```
  watch(rails.controllers) do |m|
    [
      rspec.spec.call("routing/#{m[1]}_routing"),
      rspec.spec.call("controllers/#{m[1]}_controller"),
      rspec.spec.call("acceptance/#{m[1]}")
    ]
  end

  # Rails config changes
  watch(rails.spec_helper)     { rspec.spec_dir }
  watch(rails.routes)          { "#{rspec.spec_dir}/routing" }
  watch(rails.app_controller)  { "#{rspec.spec_dir}/controllers" }

  # Capybara features specs
  watch(rails.view_dirs)     { |m| rspec.spec.call("features/#{m[1]}") }
  watch(rails.layouts)       { |m| rspec.spec.call("features/#{m[1]}") }
end
```

The guard method takes the name of one of the Guard plugins, some optional arguments (depending on the plugin), and then a block. Inside the Guard block, watched files are sent to that plugin to be processed. Right now its showing the rspec plugin, which sends changed files to RSpec to be run.

The basic idea behind Guard is this watch command, which takes a string or regular expression, or a defined method that returns one of those things, and an optional block argument. A lot of special methods are also in there, like dsl.watch_spec_files_for(rails.app_files), which effectively resolve into a series of watch commands.

The watch command is triggered when the name of a file being changed matches the string or regular-expression argument. That file name is then sent on to the plugin. If there's a block argument, you can modify the file name arbitrarily. The match data from the regular expression is the argument to the block, and the result of the block is what's sent to the plugin.

The basic idea is that if a code or spec file is changed, Guard attempts to run the related specs. So, it watches the spec helper files and reruns the entire suite if they change; but if a model file changes, it may only rerun the related spec.

The Minitest file is simpler by default. Here it is with the Spring option added and the Rails files uncommented (I slightly edited the file because the default naming conventions that Guard expects don't match what we've been doing):

```
guard :minitest, spring: true do
  watch(%r{^test/(.*)\/?test_(.*)\.rb$})
  watch(%r{^lib/(.*/)?([^/]+)\.rb$})     { |m| "test/#{m[1]}test_#{m[2]}.rb" }
  watch(%r{^test/test_helper\.rb$})      { 'test' }
```